NIVEL ALTO · EL PROBLEMA

Locks de lectura y escritura en Java

Muchos hilos leen el mismo dato y pocos lo modifican. ¿Cómo lo protegemos sin frenar a todos?



Lector + lector ✅

Inofensivo: mirar no cambia nada.
Podrían estar a la vez.

Lector + escritor ❌

El lector puede ver el dato **a medias** (mitad viejo, mitad nuevo).

Escritor + escritor ❌

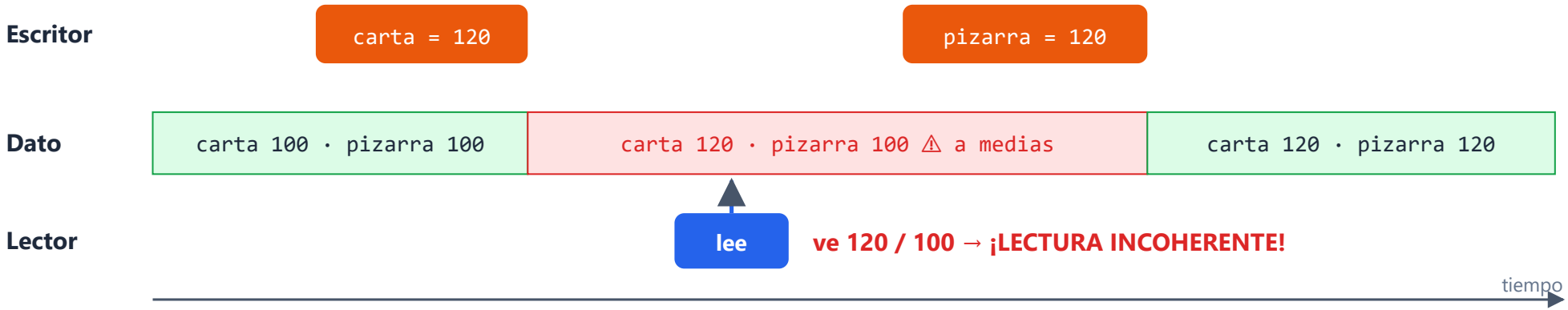
Uno **machaca** los cambios del otro.

> «Leer a la vez no es peligroso. Lo peligroso es **leer mientras alguien escribe**.»

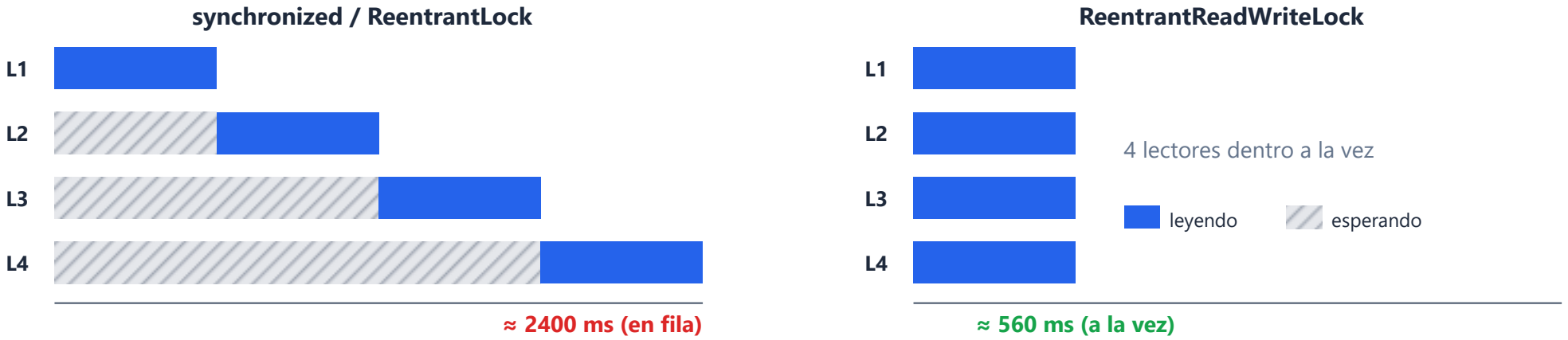
NIVEL ALTO · EL PROBLEMA

Un dato «a medias» y por qué synchronized no basta

① Sin protección: el lector mira justo entre los dos pasos del escritor



② Con un cerrojo normal se arregla… pero los lectores hacen cola entre ellos

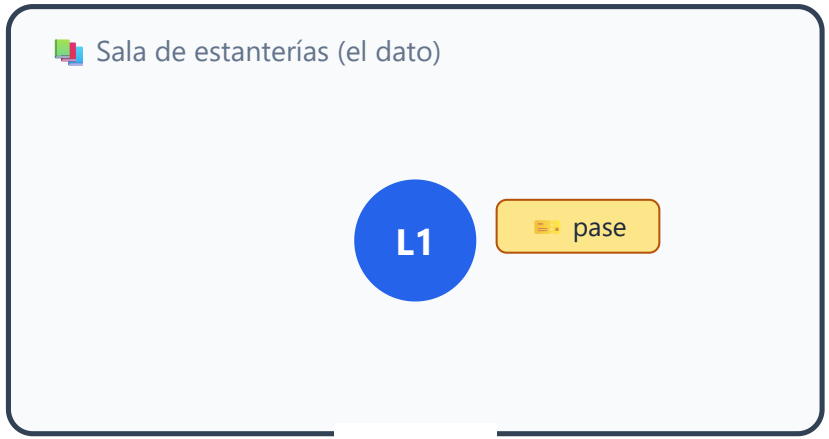


Salida real de las demos `L01` (377 lecturas incoherentes), `L02` (0 incoherentes, 2414 ms) y `L03` (0 incoherentes, 555 ms).

NIVEL ALTO · ANALOGÍA

La biblioteca: un pase o dos tipos de pase

Lock tradicional: UN pase para todo



L2 L3 L4 E

Todos en fila, aunque solo vayan a mirar

¿Por qué pueden leer varios a la vez?
Porque leer no cambia nada: diez personas leyendo el mismo libro no se molestan.

Lock lectura/escritura: DOS tipos de pase



E pase ÚNICO

Espera a que la sala esté VACÍA.
Y mientras reorganiza, no entra nadie.

¿Por qué escribir requiere estar solo?
Si alguien consulta mientras se mueven los libros, verá la estantería **a medio ordenar**.

Un objeto, dos cerrojos: readLock() y writeLock()

¿Quién puede entrar?

Dentro hay... \ llega...	Lector	Escritor
Nadie	✅ entra	✅ entra
1 o más lectores	✅ entra	❌ espera
Un escritor	❌ espera	❌ espera

readLock()
COMPARTIDO: muchos a la vez.

writeLock()
EXCLUSIVO: uno y nadie más.

ReadWriteLock = *interfaz* (solo define los 2 métodos)
ReentrantReadWriteLock = la *clase* que la implementa
Los dos cerrojos son `Lock`: `lock()`, `unlock()`, `tryLock()`...

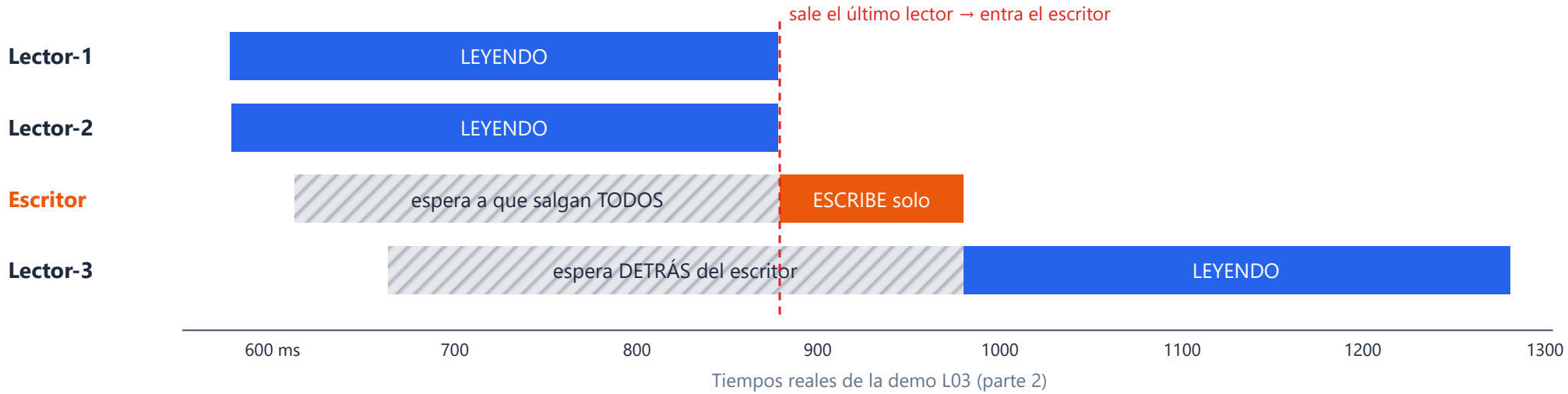
Cómo se usa

```
ReadWriteLock rw = new ReentrantReadWriteLock();
Lock lectura   = rw.readLock();
Lock escritura = rw.writeLock();

// LEER (muchos hilos a la vez)
lectura.lock();
try {
    return precio;
} finally {
    lectura.unlock();   // SIEMPRE
}

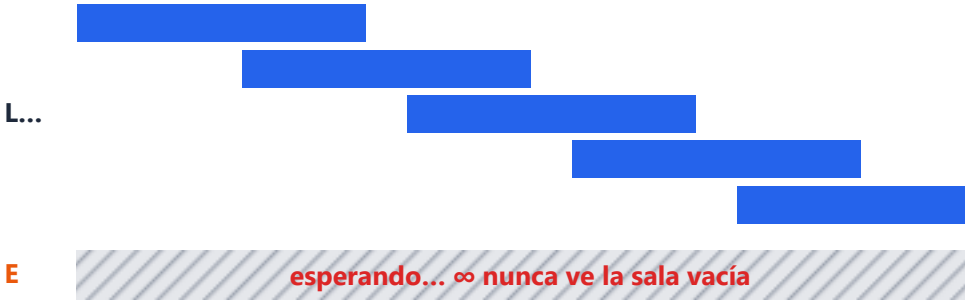
// ESCRIBIR (uno solo, sin lectores)
escritura.lock();
try {
    precio = nuevo;
} finally {
    escritura.unlock(); // SIEMPRE
}
```

El escritor espera… y los nuevos lectores no se cuelan



Tiempos reales de la demo L03 (parte 2)

Starvation: si los lectores pudieran colarse siempre…



Fairness (justicia)

Constructor	Comportamiento
`new ReentrantReadWriteLock()`	**No justo** (por defecto): más rápido, sin orden garantizado
`new ReentrantReadWriteLock(true)`	**Justo**: por orden de llegada → sin starvation, pero más lento

Subir y bajar

tengo **readLock** → pido **writeLock**	❌ **me bloqueo para siempre (me espero a mí)**
tengo **writeLock** → pido **readLock**	✅ **se puede (downgrade)**

NIVEL ALTO · LECTURA OPTIMISTA

Leer «a ojo» y comprobar después

La pizarra tiene un número de versión en la esquina. Cada vez que alguien la cambia, el número sube.



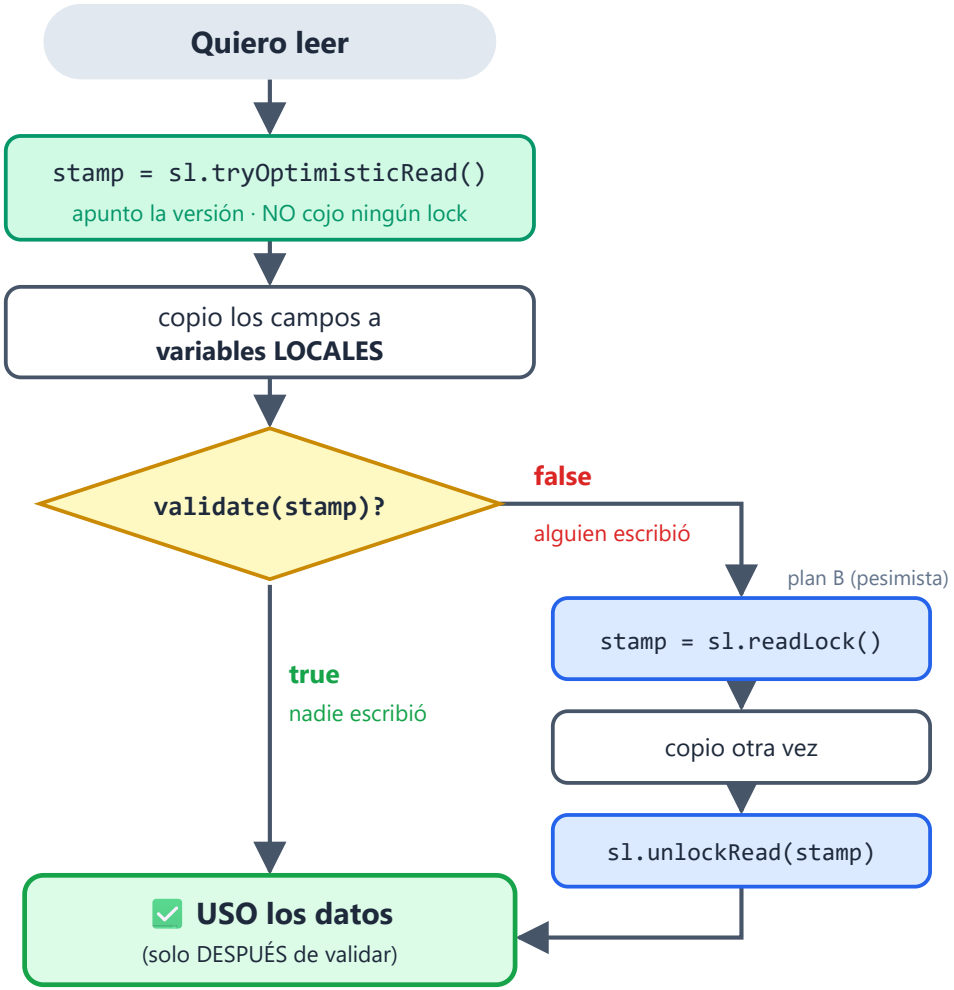
Es **optimista** porque **apuesta** a que casi nunca hay escrituras: si acierta, leer sale casi gratis.

No molesta a nadie
El que escribe **no tiene que esperar** a que yo acabe de mirar.

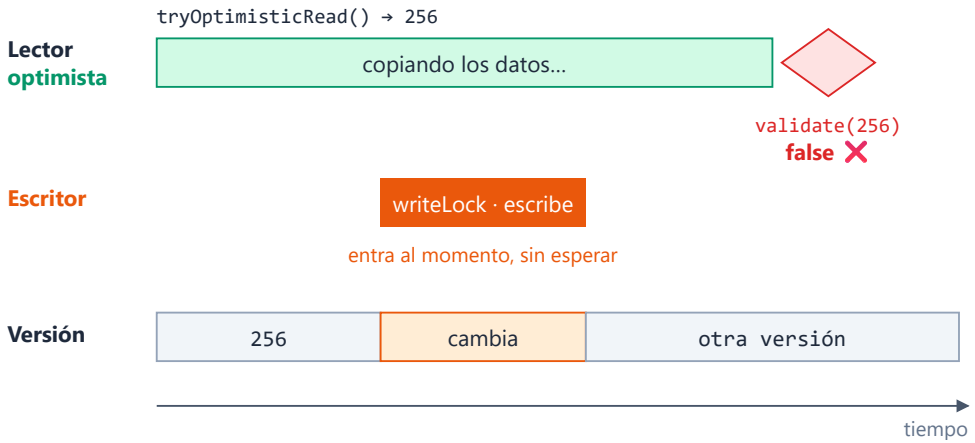
Por eso hay que comprobar
Mi foto puede haber salido **a medias** si cambiaron la pizarra durante la foto.

Compensa si…
…casi nadie escribe. Si escriben mucho, repito todo el rato y pierdo.

tryOptimisticRead() → copiar → validate(stamp)



Línea de tiempo: el escritor NO espera al lector optimista



Demo L04 (salida real): de 200 lecturas optimistas, **40** fallaron `validate`; **32** de ellas eran copias a medias. Datos incoherentes usados: **0**.

Limitaciones: hay que validar siempre · no usar los datos antes de validar · solo para leer · con muchas escrituras falla mucho · **StampedLock no es reentrante**.

`tryConvertToWriteLock(stamp)`: intenta pasar a escritura sin soltar. Devuelve el nuevo stamp o **0** si no puede.

NIVEL BAJO · COMPARATIVA

ReentrantLock vs ReentrantReadWriteLock vs StampedLock

Un pase único · dos tipos de pase · dos tipos de pase + mirar sin pase

	ReentrantLock	ReentrantReadWriteLock	StampedLock
Lectores a la vez	❌	✅	✅
Leer sin bloquear (optimista)	❌	❌	✅ **tryOptimisticRead**
¿Reentrante?	✅	✅	❌
Fairness (orden de llegada)	✅ **opcional**	✅ **opcional**	❌
Condition (await / signal)	✅	solo writeLock	❌
Subir de lectura a escritura	—	❌ **bloqueo**	✅ **tryConvertToWriteLock**
Cómo se suelta	`unlock()`	`unlock()`	`unlockRead/Write(stamp)`
Dificultad	fácil	media	**alta**
Ideal para	uso general	muchas lecturas lentas, pocas escrituras	muchísimas lecturas, escrituras raras, máximo rendimiento

Reentrante = el mismo hilo puede volver a coger el lock que ya tiene (y lo suelta las mismas veces).

Stamp = un `long` que funciona como ticket. `0` = no conseguido.

Los tres necesitan `unlock` en `finally`: no se liberan solos.

DEMO · L05_Comparativa (salida real)

Las 4 formas con el mismo trabajo

Escenario A: 4 lectores × 50 lecturas de 10 ms · 1 escritor

Sin sincronizar	548	**16** incoherentes ❌
synchronized	2414 ms	0 ✅ · lento
ReentrantReadWriteLock	567	0 ✅ · rápido
StampedLock optimista	600	0 ✅ · 20 reintentos

Lecturas que tardan algo y casi sin escrituras: los locks de lectura van **~4 veces más rápido** que synchronized y siguen siendo correctos.

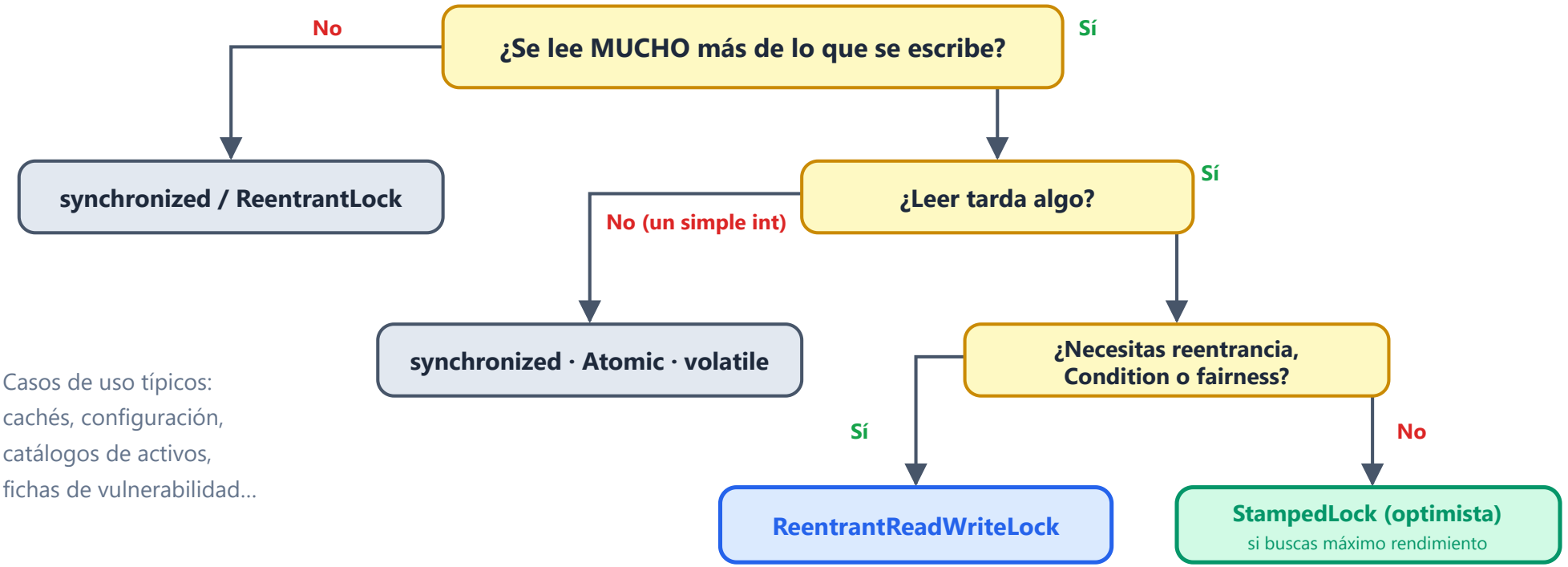
Escenario B: 8 lectores con lecturas INSTANTÁNEAS · 1 escritor

Sin sincronizar	9	**15 701** incoherentes ❌
synchronized	582 ms	0 ✅
ReentrantReadWriteLock	858 ms	**¡más lento!**
StampedLock optimista	16	0 ✅

Si leer es instantáneo, RW **empeora**: cada `readLock().lock()` modifica un contador compartido de lectores y los 8 hilos se pelean por él.

> «Rápido y mal no sirve. Primero correcto; y dentro de lo correcto, el más rápido **para mi caso**.» · Los tiempos cambian en cada ejecución: fíjate en la proporción.

¿Cuál elijo?



Casos de uso típicos:
cachés, configuración,
catálogos de activos,
fichas de vulnerabilidad…

Cuándo NO compensa

- Secciones críticas cortísimas (leer un `int`)
- Casi tantas escrituras como lecturas
- Pocos hilos o poca competencia
- Ya existe `ConcurrentHashMap`, `CopyOnWriteArrayList`…
- Datos inmutables: no hay nada que proteger

```
// Regla de oro: liberar SIEMPRE
lock.lock();          // fuera del try
try {
    ...
} finally {
    lock.unlock();    // aunque haya excepción
}
// Si no: el lock queda cogido para siempre
// y el programa se congela SIN ningún error.
```